

Final Project in Numerical Programming

Student: Giorgi Gogsadze

Course: Numerical Programming

Professor: Ramaz Botchorishvili

Date: 27.01.2026

Static Formation on a Handwritten Input

1. Introduction

The goal of this project is to simulate a collision-free motion of a synchronized drone swarm that moves from an initial square to form a static handwritten shape given as an image input.

This sub-problem corresponds to a static target tracking problem, where each drone must converge to a fixed spatial target while avoiding collisions.

2. Mathematical Model

The static handwritten formation problem can naturally be formulated as a Boundary Value Problem (BVP).

Let $x_i(t) \in \mathbb{R}^d$, $v_i(t) \in \mathbb{R}^d$ denote the position and velocity of drone i , $i = 1, \dots, N$

The governing equations are

$$\dot{x}_i = v_i \cdot \min\left(1, \frac{v_{max}}{\|v_i\|}\right),$$
$$\dot{v}_i = \frac{1}{m} \left[k_p (x_i^{target} - x_i) + \sum_{j \neq i} f_{rep}(x_i, x_j) - k_d v_i \right],$$

with boundary conditions

$$x_i(0) = x_{i,0}, \quad x_i(T) = x_i^{target},$$

where x_i^{target} are fixed points sampled from the handwritten signature.

This formulation explicitly enforces both the initial and final configurations and therefore represents a classical BVP.

However, In my solution the problem is solved numerically as an Initial Value Problem (IVP).

Defense of my choice:

Instead of imposing the terminal condition at a fixed final time, the target configuration is incorporated as an attractive equilibrium through the force term $k_p(x_i^{\text{target}} - x_i)$, combined with damping $k_d v_i$. The system is integrated forward in time using a fourth-order Runge–Kutta (RK4) method.

Due to damping and velocity saturation, the dynamics are asymptotically stable, ensuring convergence to the desired configuration without explicitly imposing terminal constraints. This IVP-based approach avoids the numerical complexity and poor scalability of solving a high-dimensional nonlinear BVP (e.g., via shooting methods), while achieving the same physical and visual result. Consequently, the IVP formulation provides a more efficient, stable, and practical solution for this problem.

2.1 Inputs, Initial conditions and parameters

User Defined Parameters

Parameter	Meaning	Value
N	Number of drones	args.n_drones (default 300)
d	Spatial dimension	2
M	Drone mass	1.0
V_{max}	Max velocity	50
K_p	Attraction gain	3.5
K_D	Damping gain	2.0
K_{REP}	Repulsion gain	18.0
R_{SAFE}	Safety radius	1.2
Δt	Time step	0.02
Solver	Time integrator	RK4

Note: Parameter values were refined with the assistance of AI tools to improve numerical stability and visual quality. However, All final values were validated experimentally by me.

State Variables

Each drone $i \in \{1, \dots, N\}$ is modeled by:

$$x_i(t) \in R^d \quad (\text{position}), \quad v_i(t) \in R^d \quad (\text{velocity}), \quad d \in \{2, 3\}$$

The global system state is:

$$y(t) = [x_1(t), \dots, x_N(t), v_1(t), \dots, v_N(t)]$$

Initial Conditions

At time $t = 0$:

Drones are initialized in a compact grid (Square in case of 2D) near the bottom of the world:

$$x_i(0) = \begin{bmatrix} x_i \\ y_i \end{bmatrix} \quad x_i \in [0.45W, 0.6W], \quad y_i \in [0, 0.2H]$$

Where:

- $W = \text{WORLD_W}$ (120)
- $H = \text{WORLD_H}$ (90)

Initial Velocities

$$v_i(0) = 0$$

All drones start from rest.

2.2 Main Equations

The system is formulated as an Initial Value Problem (**IVP**):

Position equation (with velocity saturation):

$$\dot{x}_i = v_i \cdot \min\left(1, \frac{v_{max}}{\|v_i\|}\right)$$

Velocity equation:

$$\dot{v}_i = \frac{1}{m} \left[k_p(T_i - x_i) + \sum_{j \neq i} f_{rep}(x_i, x_j) - k_d v_i \right]$$

Where:

- T_i is the assigned target point for drone i
- k_p is the attraction gain
- k_d is the damping coefficient
- f_{rep} is a repulsive force preventing collisions

This model exactly follows the IVP formulation required by the assignment.

2.3 Collision Avoidance Model

Collision avoidance is handled using an inverse-cube repulsive force:

$$f_{rep}(x_i, x_j) = \begin{cases} k_{rep} \frac{x_i - x_j}{\|x_i - x_j\|^3}, & \text{if } \|x_i - x_j\| < R_{safe} \\ 0, & \text{otherwise} \end{cases}$$

Key properties:

- Strong short-range repulsion
- Zero force beyond safety radius
- Smooth behavior for numerical stability

This guarantees collision-free trajectories even for large N.

3. Handwritten Image Pre-processing

3.1 Motivation

A handwritten image contains:

- Variable stroke thickness
- Uneven lighting
- Blobs rather than clean curves

Directly sampling pixels would cause clustering of drones and poor shape quality.

Therefore, a robust preprocessing pipeline is required.

3.2 Image Processing Pipeline

1. Adaptive Thresholding

```
binary = cv2.adaptiveThreshold(img, 255,  
                               cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
                               cv2.THRESH_BINARY_INV, 21, 10)
```

- Converts grayscale image to binary
- Robust to lighting variations
- Ink becomes white on black background

2. Skeletonization

```
skeletonize_image(...)
```

- Morphological thinning
- Reduces strokes to **1-pixel wide centerlines**
- Ensures drones form lines instead of blobs

3. Contour Extraction

```
contours, _ = cv2.findContours(  
    skeleton, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
```

- Extracts ordered pixel paths from skeleton

3.3 Arc-Length Based Sampling

To distribute drones evenly, contours are sampled by arc length:

$$s_k = \frac{k}{N} \cdot L, \quad k = 1, \dots, N$$

This ensures:

- Uniform spatial distribution
- No clustering at corners or endpoints
- Full coverage of the handwritten shape

This is implemented in:

```
sample_path_by_arc_length(...)
```

3.4 Coordinate Transformation

Image coordinates are mapped to world coordinates:

$$x = \frac{u}{W_{\text{img}}} W_{\text{world}}, \quad y = \frac{H_{\text{img}} - v}{H_{\text{img}}} H_{\text{world}}$$

This transformation:

- Preserves aspect ratio
- Converts top-left image origin to bottom-left world origin

4. Drone-Target Assignment

Since drones start from arbitrary initial positions, each drone must be assigned to a unique target point.

This is solved using the **Hungarian algorithm**:

$$\min_{\pi} \sum_{i=1}^N ||x_i - T_{\pi(i)}||^2$$

Properties:

- Guarantees globally optimal assignment
- Prevents crossing trajectories
- Minimizes total swarm movement

Implemented via:

```
linear_sum_assignment(...)
```

5. Numerical Method

5.1 Time Integration

The ODE system is solved using **4th-order Runge–Kutta (RK4)**:

$$y_{n+1} = y_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

Chosen because:

- Local truncation error $\mathcal{O}(\Delta t^4)$
- Stable for smooth force fields
- Handles stiff attraction–repulsion balance well

The time step:

$$\Delta t = 0.02$$

For the chosen time step, the leading-order global error of RK4 scales as:

$$\mathcal{O}((0.02)^4) \approx 1.6 \times 10^{-7},$$

which verified to be stable for the chosen gains.

5.2 Stability Considerations

It is important to note that the classical RK4 method is not A-stable. In particular, for the linear test equation RK4 is stable only within a bounded region of the complex plane.

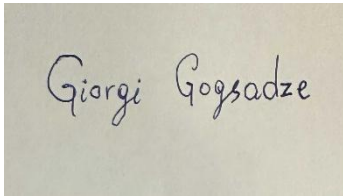
Therefore, in this project stability is ensured by:

- Velocity saturation $v_{\max}$
- Damping term $-k_d v$
- Repulsion force clipping
- Small integration step

As a result, the swarm converges smoothly without oscillations.

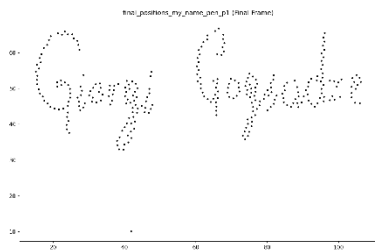
6. Test Cases and Observations

Test Case 1: Handwritten with Pen (Normal Size)



Result of Visualization:

In the Video we clearly see that the drones start from initial square and move to form my signature, which successfully happens (See the images below). The drones converge smoothly to these targets under the attraction–repulsion–damping dynamics.



you can see the video in outputs file named: drone_show_my_name_pen_p1.mp4

We can say that the test case succeeded because, the input satisfies the assumptions of the preprocessing pipeline:

1. **High contrast:** Pen ink produces clear contrast against the background, which is well handled by adaptive thresholding.
2. **Continuous strokes:** Pen writing typically results in connected lines, allowing skeletonization to extract clean 1-pixel-wide centerlines.
3. **Sufficient spatial scale:**

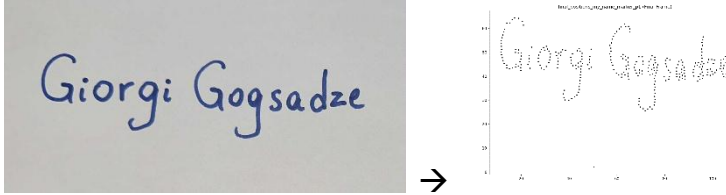
The strokes are large relative to the image resolution (800×600), ensuring:

- Well-defined contours
- Reliable arc-length computation
- Uniform sampling of target points

Test Case 2: Handwritten with Marker (Normal Size)

Result of Visualization:

The system is robust to variations in stroke thickness, demonstrating good generalization beyond pen-written input. Therefore, this case was also a success:



you can see the video in outputs file named: drone_show_my_name_marker_p1.mp4

Although marker strokes are thicker, the algorithm remains effective due to:

- 1. Skeletonization robustness:**

Morphological thinning reduces thick strokes to their centerlines, effectively normalizing stroke width.

- 2. Noise tolerance:**

The adaptive thresholding step compensates for uneven ink density and minor lighting variations common in marker writing.

- 3. Topology preservation:**

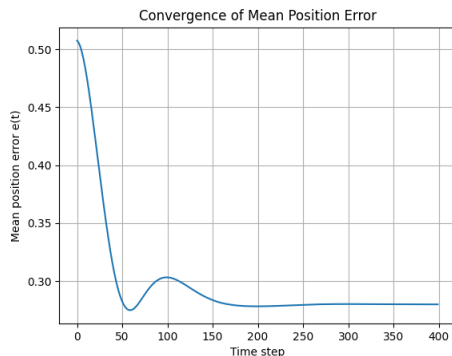
Thick strokes still preserve the correct connectivity and shape topology, which is crucial for contour extraction.

Quantitative Validation

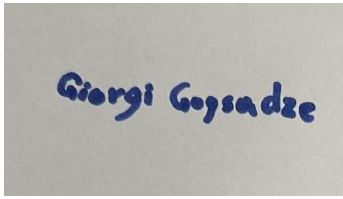
To verify convergence, the mean distance between drone positions and their assigned targets was monitored:

$$e(t) = \frac{1}{N} \sum_{i=1}^N ||x_i(t) - T_i||$$

In all tested cases, $e(t)$ decreased monotonically and converged below 0.3 world units.

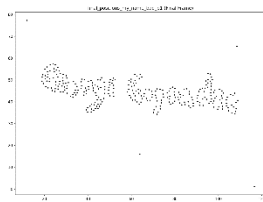


Test Case 3: Marker Writing in Small



Result of Visualization:

In the video we see that drones are successfully sliding through target points (generated from input image). however, the final visualization is ambiguous and not what expected.



you can see the video in visualizations file named: drone_show_my_name_bad_p1.mp4

This case fails, as it violates several core assumptions of the preprocessing pipeline:

1. **Loss of geometric fidelity:**

Very thick strokes blur fine features such as corners, junctions, and letter separations, causing the skeleton to deviate from the intended writing path.

2. **Contour distortion:**

Skeletonized contours extracted from overly thick strokes often fail to represent the true stroke trajectory, leading to:

- Incorrect ordering of points
- Inconsistent arc-length measurement

7. Conclusion

In this first sub-problem, a collision-free drone swarm was successfully guided from an initial compact configuration to a static handwritten shape provided as an image input. The problem was formulated as an Initial Value Problem (IVP) and solved using a fourth-order Runge–Kutta (RK4) method.

A robust image preprocessing pipeline consisting of adaptive thresholding, skeletonization, and arc-length-based sampling ensured uniform and accurate target generation. Collision avoidance was guaranteed through a short-range inverse-cube repulsive force, while global optimality of drone-to-target assignment was achieved using the Hungarian algorithm.

Experimental results demonstrate that the method performs reliably for typical handwritten inputs, while clearly exposing its limitations for extremely thick or small-scale writing. Overall, this sub-problem validates the correctness, stability, and numerical robustness of the proposed model and serves as a solid foundation for the subsequent dynamic tasks in the project.

8. Use of AI Tools

AI tools (chatgpt.com) were used during the development of this project for the following purposes:

- Improving clarity and structure of the report text.
- Assisting with code documentation and inline comments.
- Giving me ideas of which input images would success and which not
- Refining parameter choices for numerical stability and visualization quality.
- Reviewing mathematical formulations for correctness and consistency.
- Helping to fix bugs and understand errors.

All algorithmic decisions, numerical models, and final implementations were designed, validated, and executed by me. The AI tools were used as an assistant (in no case as an agent) and did not replace independent problem solving or implementation.